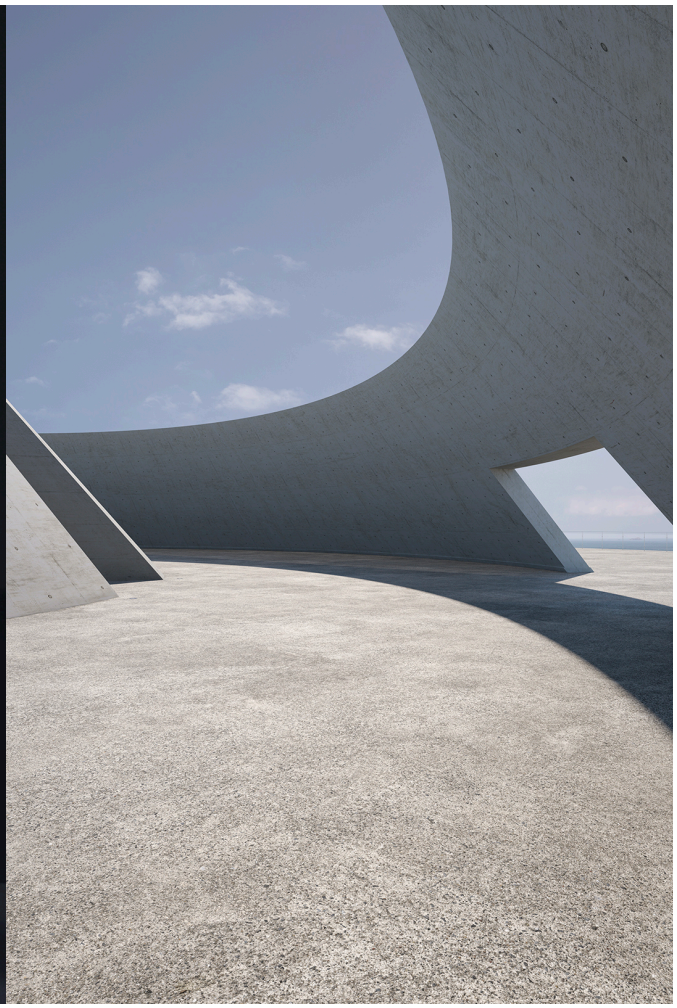
B+树索引的原理与实现

邹兆年 教授
哈尔滨工业大学计算学部

2025年全国大学生计算机系统能力大赛数据库管理系统设计赛技术培训会，2025.05.25

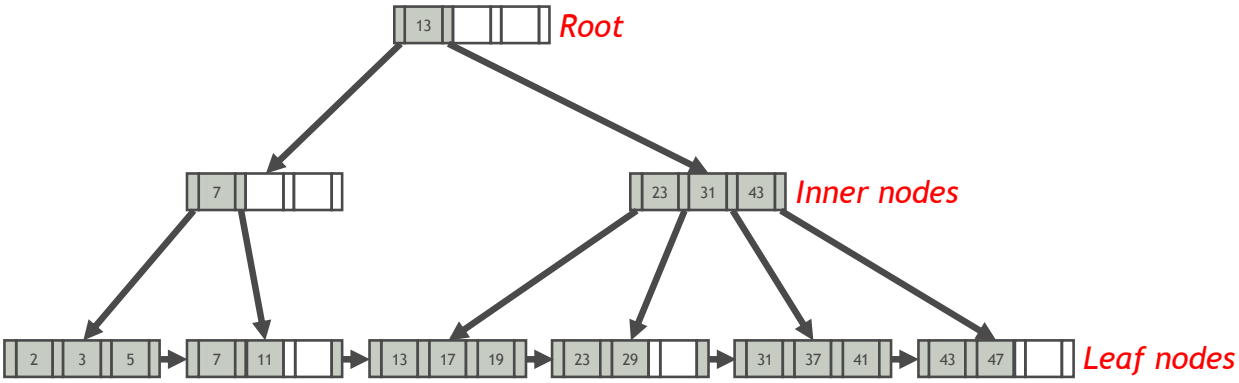
B+树结构



B+树(B+ Trees)

B+树是一棵M路平衡搜索树，它具有以下性质:

- B+树是一棵完美的平衡树，所有叶节点都在同一层上
- 除根节点外，每个节点至少"半满"，即$M/2-1 \leq \#keys \leq M-1$[3]
- 每个节点恰好放入1个页

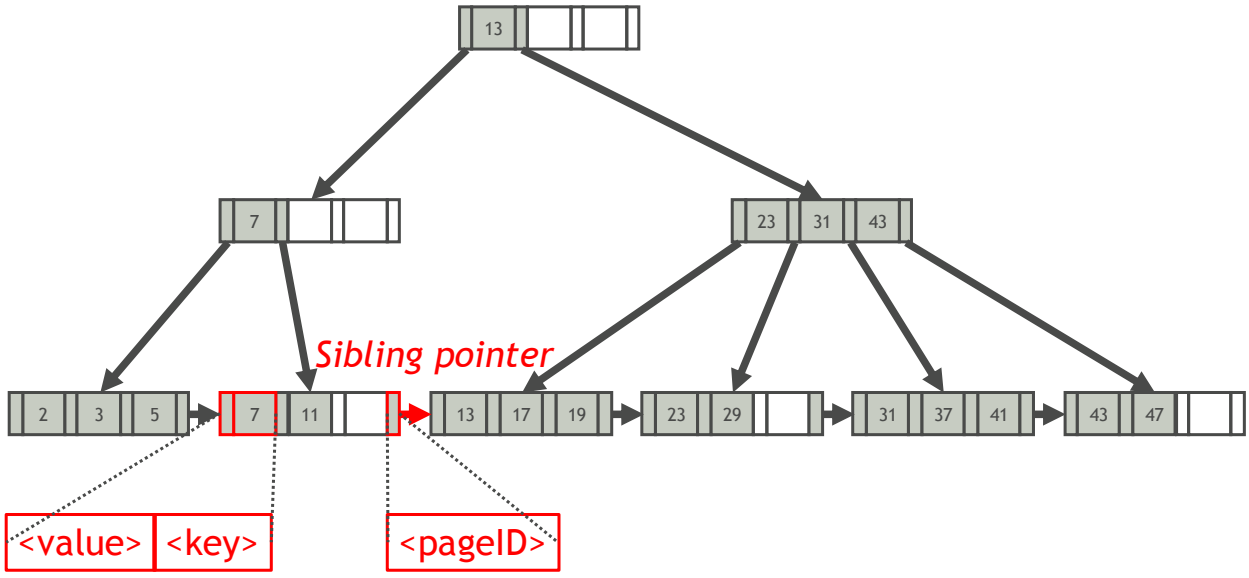


[3]Raghu Ramakrishnan, Johannes Gehrke. Database Management Systems, 3rd Edition. 2003

B+树的叶节点(Leaf Nodes)

每个叶节点包含一个索引项数组和一个指向右侧兄弟叶节点的指针(右兄弟节点的页号)

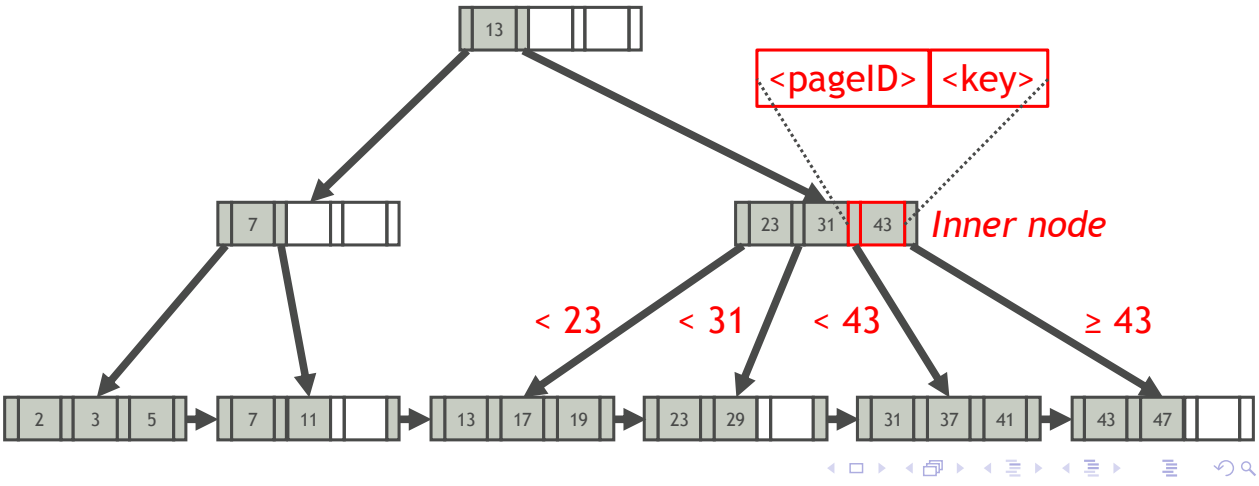
- 索引项数组通常按索引键排序



B+树内节点(Inner Nodes)

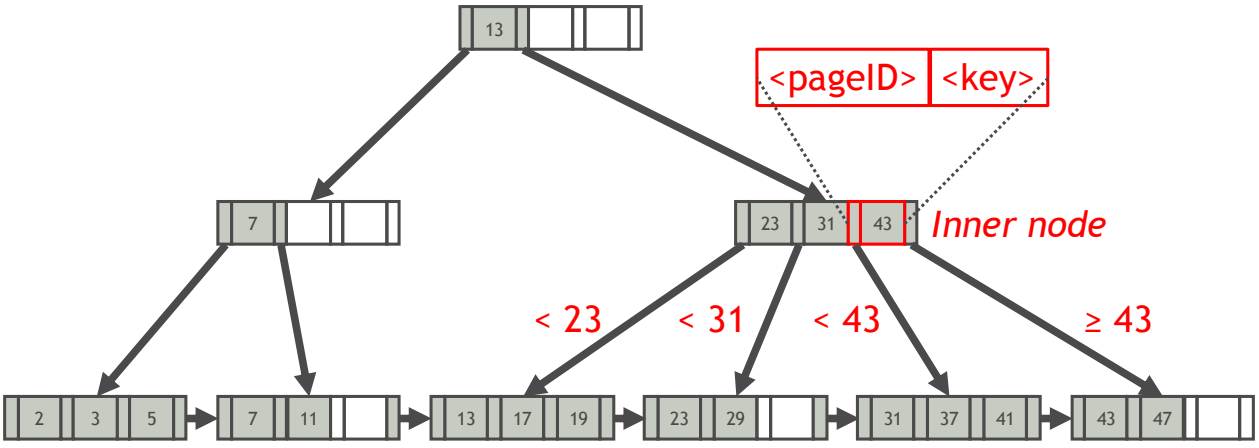
每个内节点包含一个键数组Key和一个指向儿子节点的指针的数组Ptr

- Key中有k的非空键值当且仅当Ptr中有$k+1$个非空指针
- Key中的键值排序
- $Ptr[0]$指向的子树中的键值$<Key[0]$
- $Ptr[k+1]$指向的子树中的键值$\geq Key[k]$
- $Key[i-1] \leq Ptr[i]$指向的子树中的键值$<Key[i]$



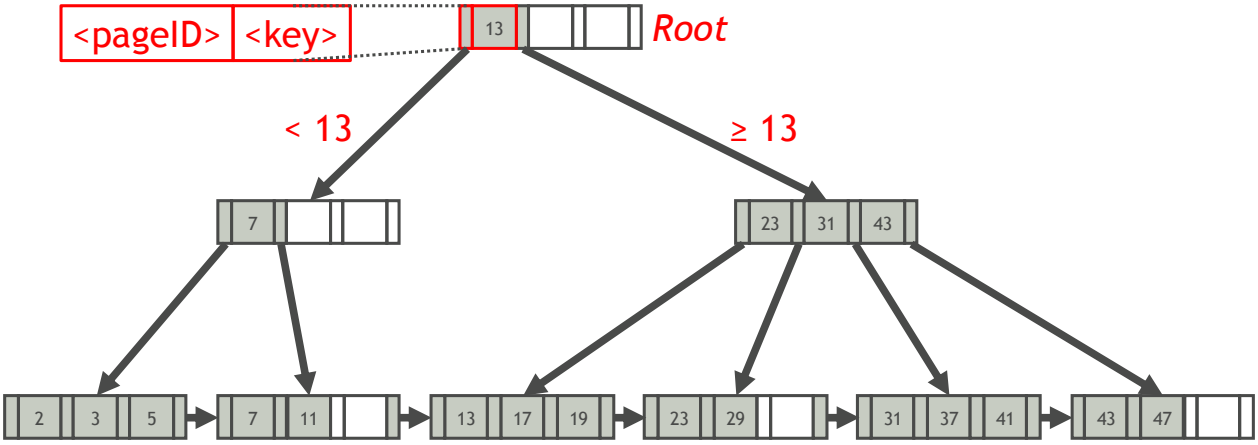
B+树内节点(Inner Nodes)

对于$i \geq 1$，$Key[i-1]$存放的是$Ptr[i]$指向的子树叶节点中的最小键值

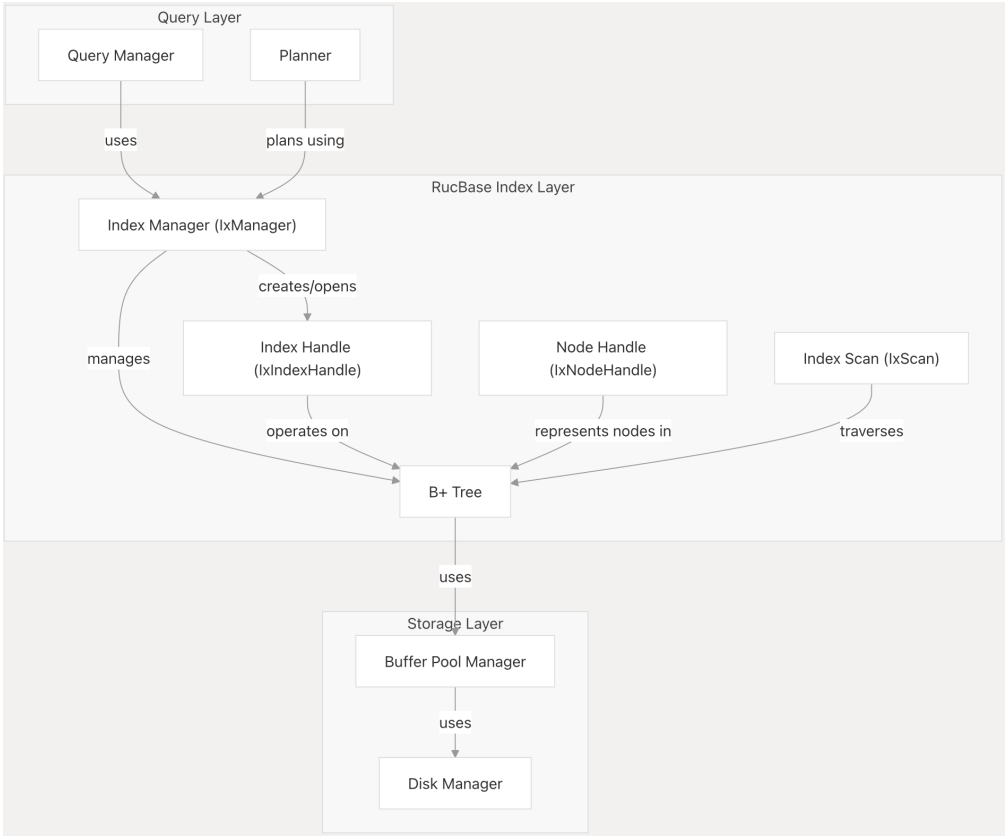


B+树的根节点(Root Node)

根节点和内节点的内部结构相同，但不要求“半满”(根节点中包含至少1个键即可)



Rucbase中B+树索引



Rucbase中B+树节点的定义ix_node_handle.h

```
class IxNodeHandle {
friend class IxIndexHandle;
friend class IxScan;

private:
const IxFileHdr *file_hdr;  // 用到了file_hdr的keys_size, col_
Page *page;

IxPageHdr *page_hdr; // page->data的第一部分，指针指向首地址，
后续占用长度为sizeof(IxPageHdr)
char *keys; // page->data的第二部分，指针指向首地址，后续占用
长度为file_hdr->keys_size，每个key的长度为file_hdr->col_len
Rid *rids; // page->data的第三部分，指针指向首地址，每个rid的
长度为sizeof(Rid)
};
```

▸ 演示

Rucbase中B+树节点的声明ix_node_handle.h

```
struct IxPageHdr {
page_id_t next_free_page_no;
page_id_t parent;  // its parent's page_no
int num_key;  // # current keys (always equals to #child - 1)
插入的keys数量，key_idx ∈ [0,num_key)
bool is_leaf;
page_id_t prev_leaf;  // previous leaf node's page_no, effecti
page_id_t next_leaf;  // next leaf node's page_no, effective c
};
```

▸ 演示

Rucbase中B+树的声明ix_index_handle.h

```
class IxIndexHandle {
friend class IxScan;
friend class IxManager;

private:
DiskManager *disk_manager_;
BufferPoolManager *buffer_pool_manager_;
int fd_;
IxFileHdr file_hdr_;  // 存了root_page，但root_page初始化为2（存FILE_HDR_PAGE，第1页存LEAF_HEADER_PAGE）
std::mutex root_latch_;  // 用于索引并发（请自行选择并发粒度 在 Tree级 或 Page级 ）
};
```

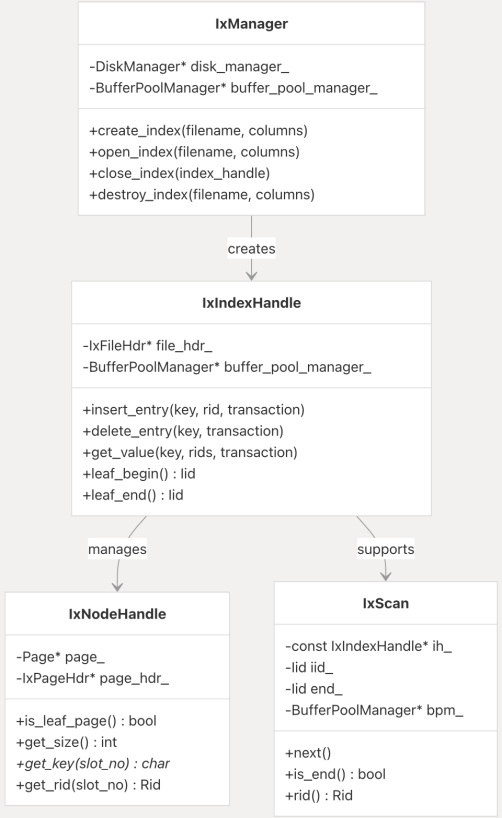
▸ 演示

Rucbase中B+树的声明ix_index_handle.h

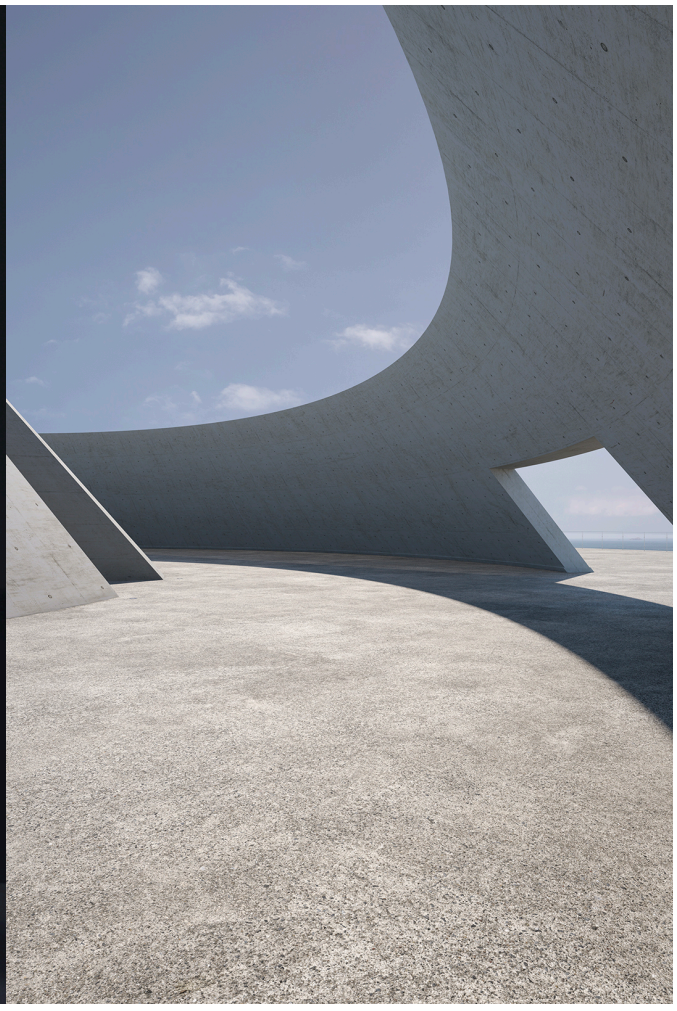
```
struct IxFileHdr {
page_id_t first_free_page_no;
int num_pages;          // disk pages
page_id_t root_page;  // root page no
ColType col_type;
int col_len;        // ColMeta->len
int btree_order;  // children per page 每个结点最多可插入的键值对数量
int keys_size;  // keys_size = (btree_order + 1) * col_len
page_id_t first_leaf;  // 在上层IxManager的open函数进行初始化，初始化为root page_no
page_id_t last_leaf;
};
```

▸ 演示

Rucbase中B+树索引相关的类



B+树操作

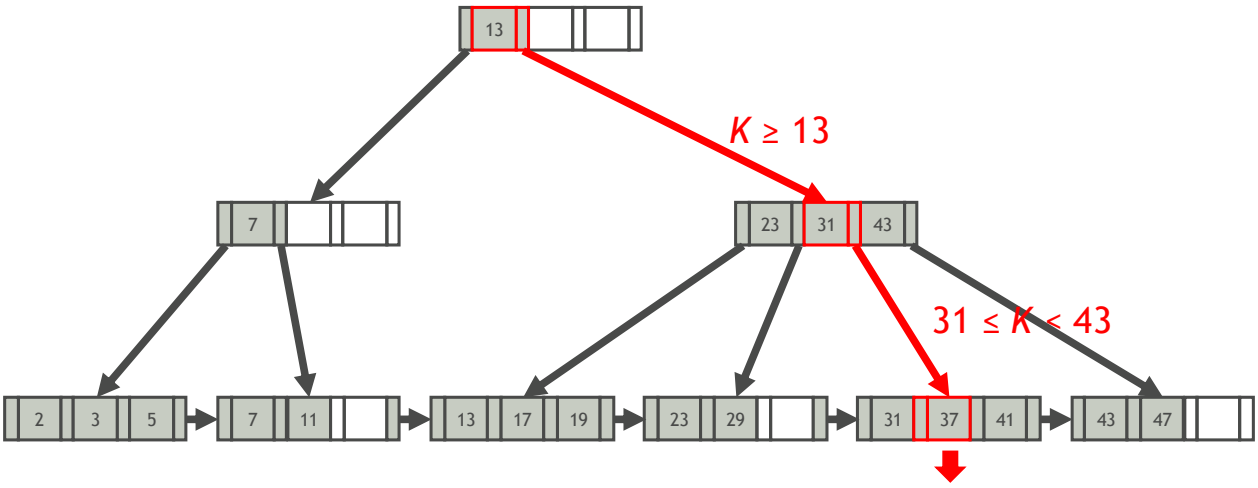


查找索引项

查找键为K

1. 在内节点的引导下，找到K属于哪个叶节点
2. 在该叶节点中查找键值为K的索引项

例: $K = 37$

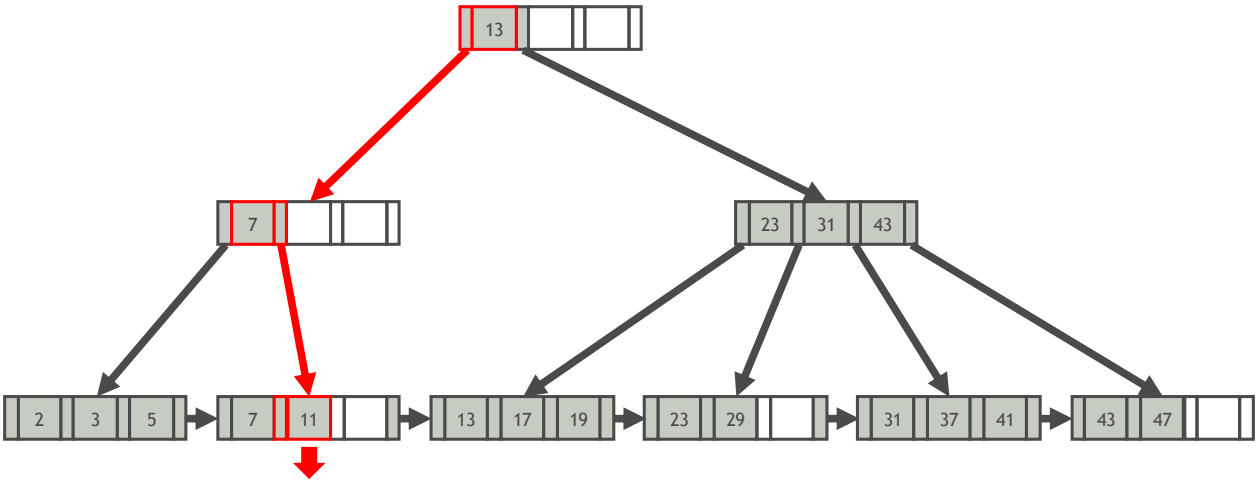


区间查询

查找键在区间$[L, U]$内的全部索引项

1. 找到具有大于等于L的最小键的索引项E
2. 扫描E右侧的索引项，如果键$\leq U$，则输出；否则，终止

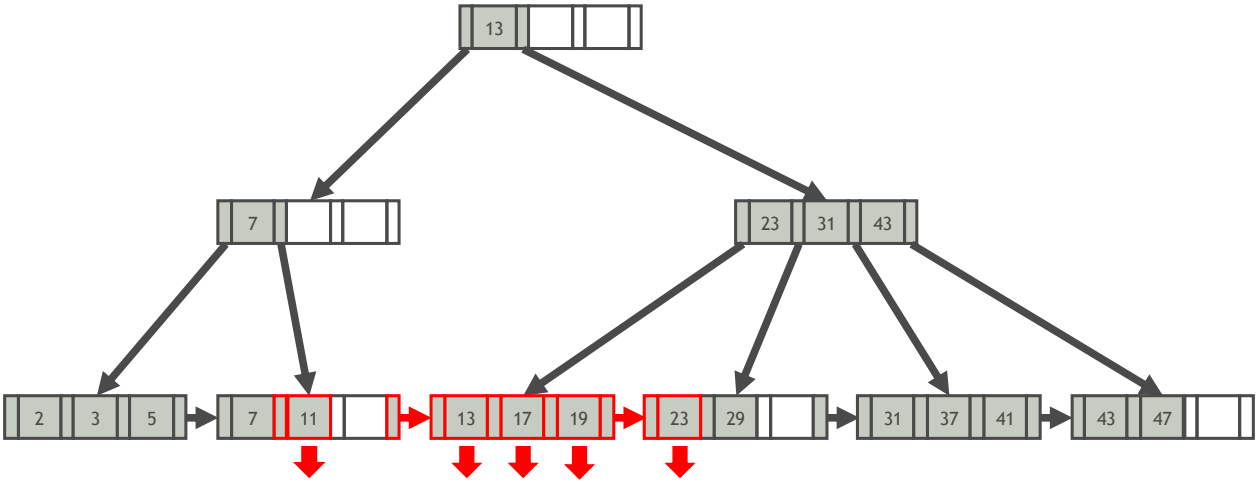
例: $K \in [10, 25]$



步骤1/2

区间查询(续)

例: $K \in [10, 25]$



步骤2/2

Rucbase中B+树查找函数

```
IxNodeHandle *IxIndexHandle::FindLeafPage(
const char *key, Operation operation,
Transaction *transaction);

bool IxIndexHandle::GetValue(
const char *key, std::vector<Rid> *result,
Transaction *transaction);

// 在当前node中查找第一个>=target的key_idx
int IxNodeHandle::lower_bound(const char *target) const;

// 在当前node中查找第一个>target的key_idx
int IxNodeHandle::upper_bound(const char *target) const;
```

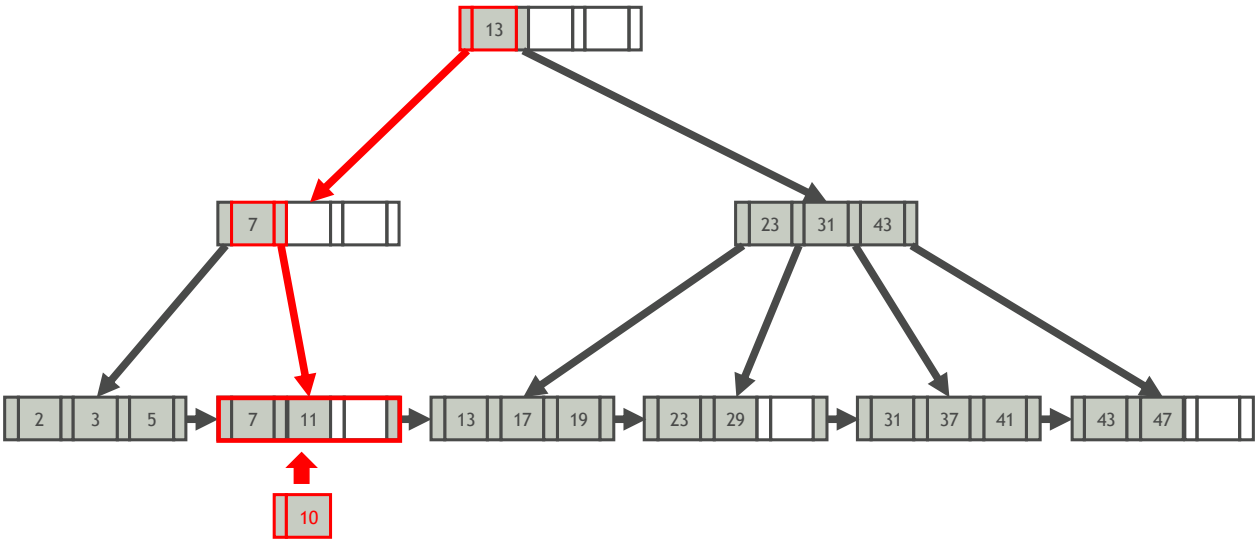
插入索引项

插入键为K的索引项

1. 找到K应在的叶节点L
2. 将索引项插入L
3. 如果L不溢出，则插入完成；否则，分裂(split)L

插入索引项(续)

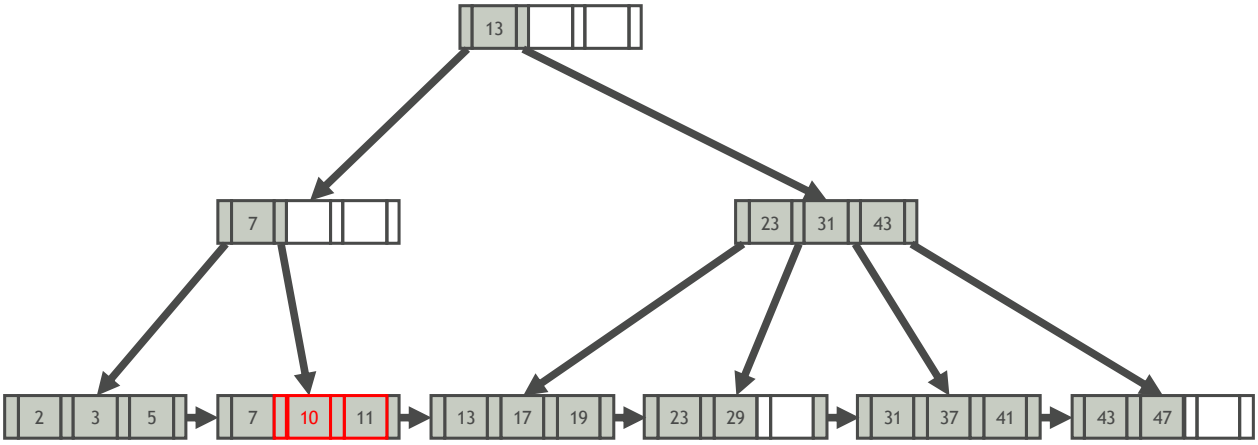
例1: $K = 10$ (无节点分裂)



步骤1/2

插入索引项(续)

例1: $K = 10$ (无节点分裂)



步骤2/2

叶节点分裂

分裂叶节点L

1. 创建一个新的叶节点L_2
2. 将L中的索引项平分，前一半留在L中，后一半移入L_2中
3. 将L_2中最小的键存入“中间键(middle key)”变量
4. 在叶节点链表中，将L_2插到L的右边
5. 在L的父节点N中插入middle key及指向L_2的指针
6. 如果N不溢出，则完成对L的分裂；否则，继续分裂N

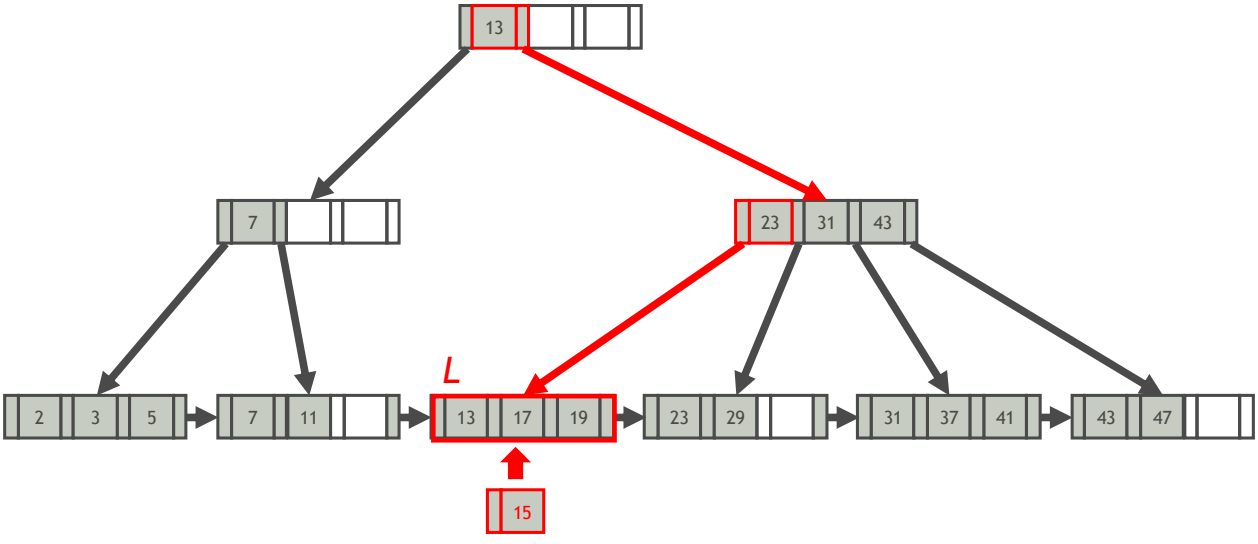
内节点分裂

分裂内节点N

1. 创建一个新的内节点N_2
2. 将N中的指针平分，前一半留在N中，后一半移入N_2中
3. 将N中多余的键存入“中间键(middle key)”变量
4. 如果N是根节点，则创建一个新的根节点N'，并在N'中插入一个指向N的指针
5. 在N的父节点N'中插入middle key及指向N_2的指针
6. 如果N'不溢出，则完成对N的分裂；否则，继续分裂N'

插入索引项(续)

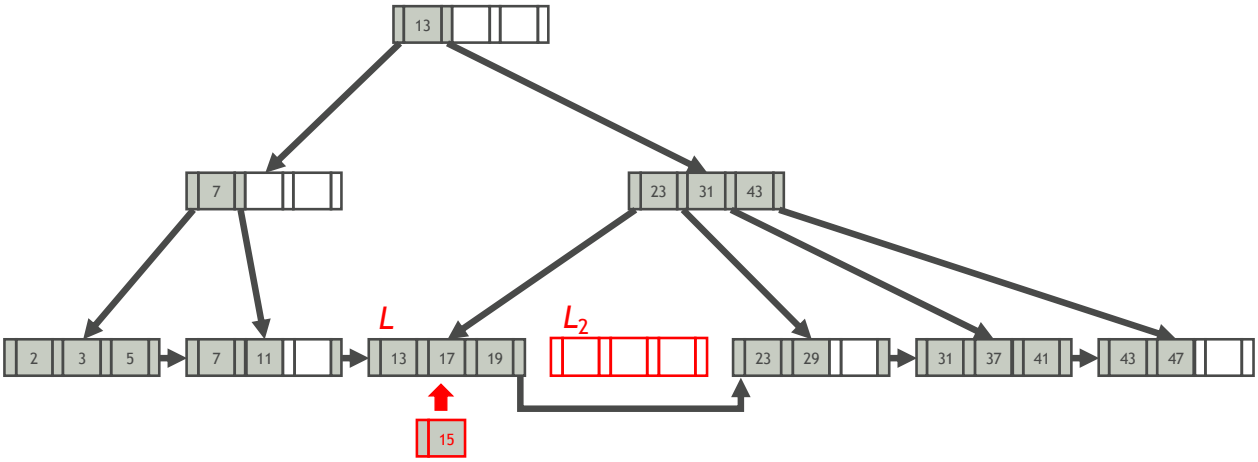
例2: $K = 15$ (有节点分裂)



步骤1/7

插入索引项(续)

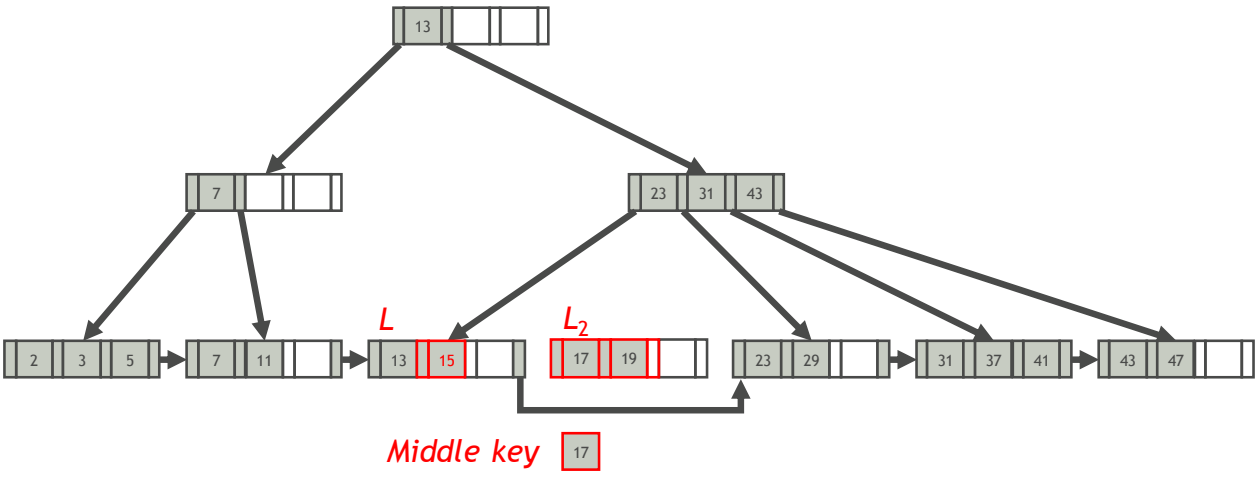
例2: $K = 15$ (有节点分裂)



步骤2/7

插入索引项(续)

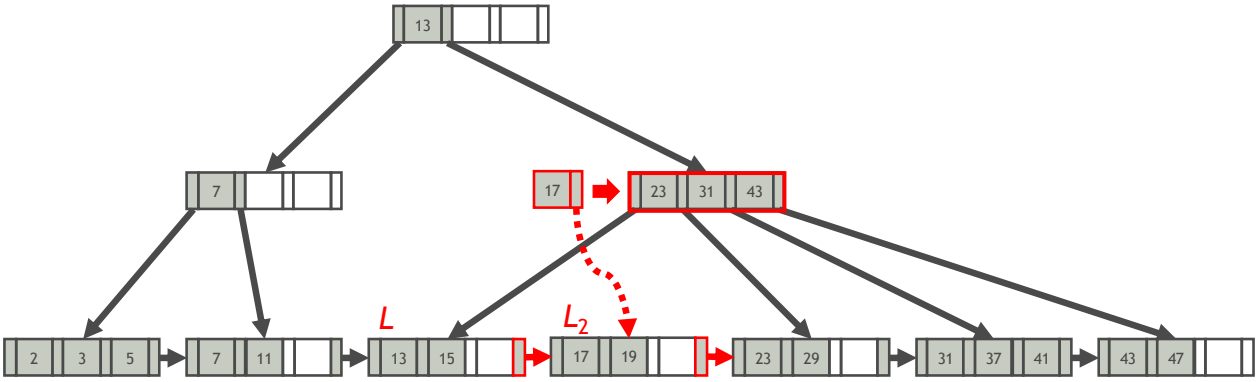
例2: $K = 15$ (有节点分裂)



步骤3/7

插入索引项(续)

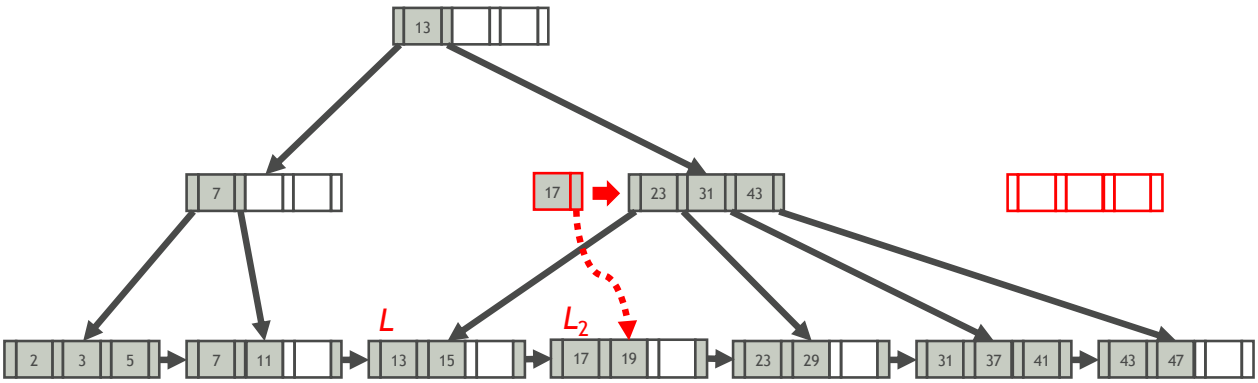
例2: $K = 15$ (有节点分裂)



步骤4/7

插入索引项(续)

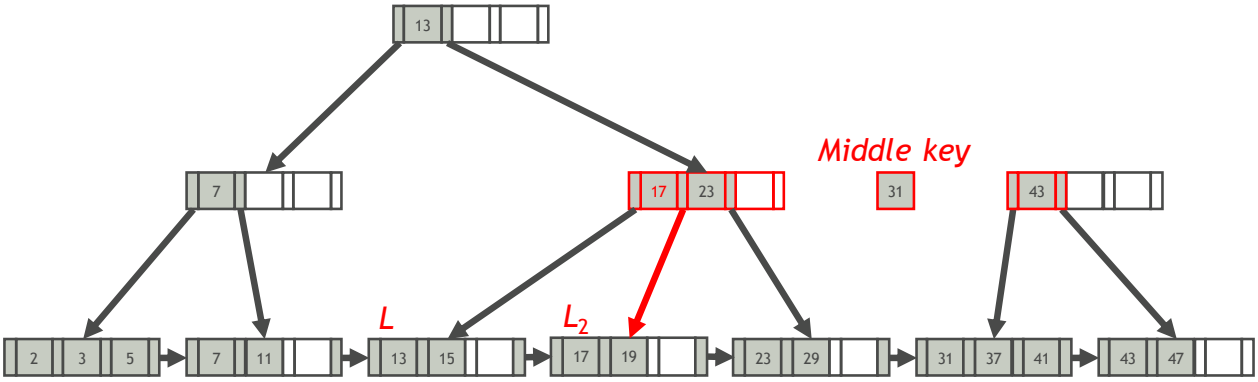
例2: $K = 15$ (有节点分裂)



步骤5/7

插入索引项(续)

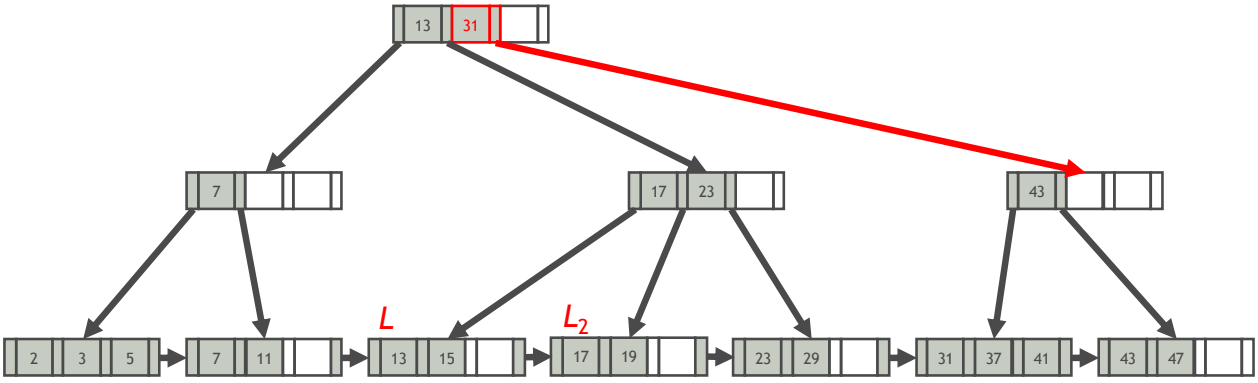
例2: $K = 15$ (有节点分裂)



步骤6/7

插入索引项(续)

例2: $K = 15$ (有节点分裂)



步骤7/7

Rucbase中B+树插入函数

```
bool IxIndexHandle::insert_entry(
const char *key, const Rid &value,
Transaction *transaction);

IxNodeHandle *IxIndexHandle::Split(IxNodeHandle *node);

void IxIndexHandle::InsertIntoParent(
IxNodeHandle *old_node, const char *key,
IxNodeHandle *new_node, Transaction *transaction);
```

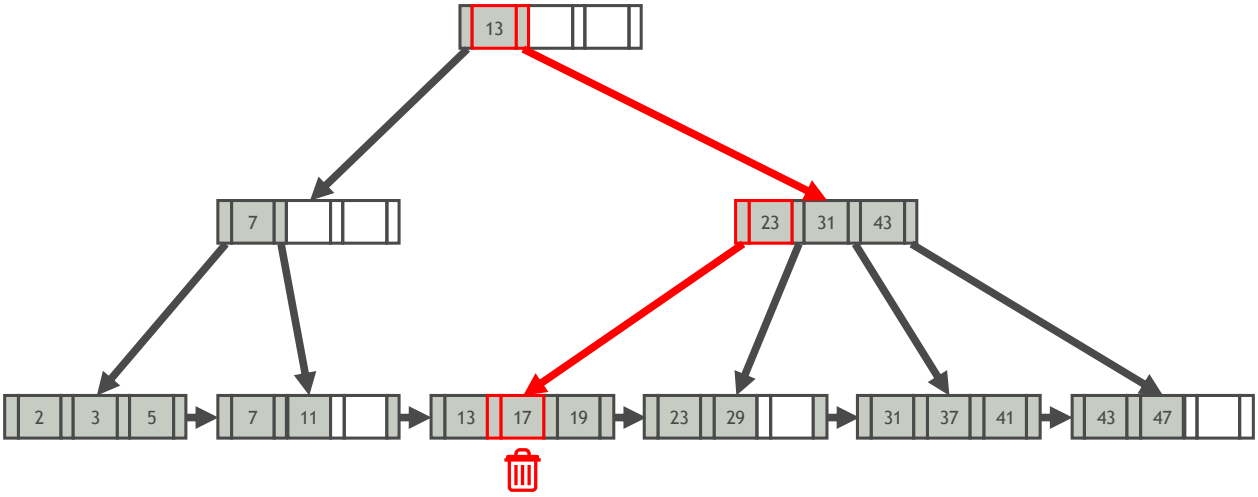

删除索引项

删除键为K的索引项

1. 找到K所在的叶节点L
2. 从L中删除键为K索引项
3. 如果L至少半满，则完成删除；否则，处理L，使L至少半满

删除索引项(续)

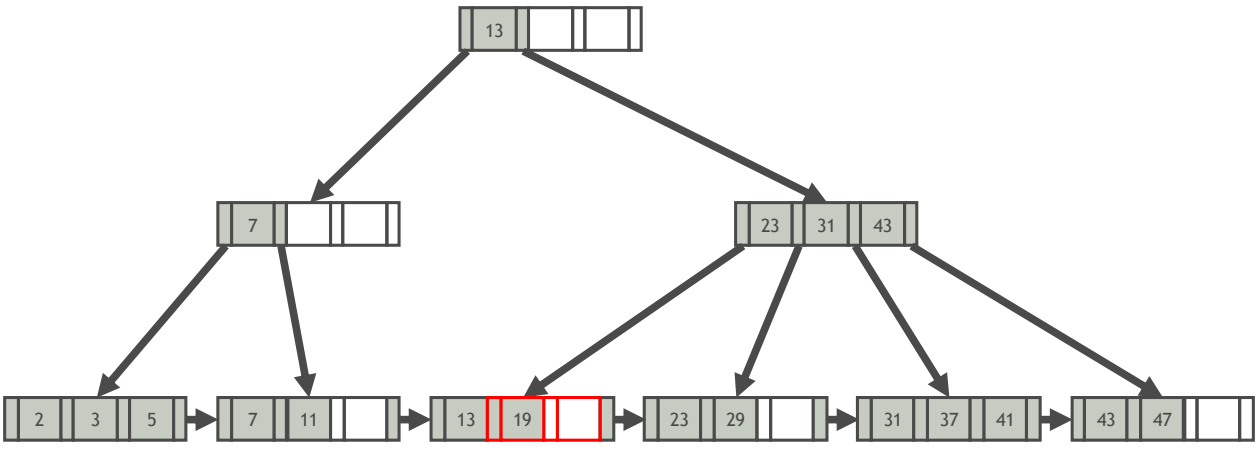
例1: $K = 17$ (没有键重分布及节点合并)



步骤1/2

删除索引项(续)

例1: $K = 17$ (没有键重分布及节点合并)



步骤2/2

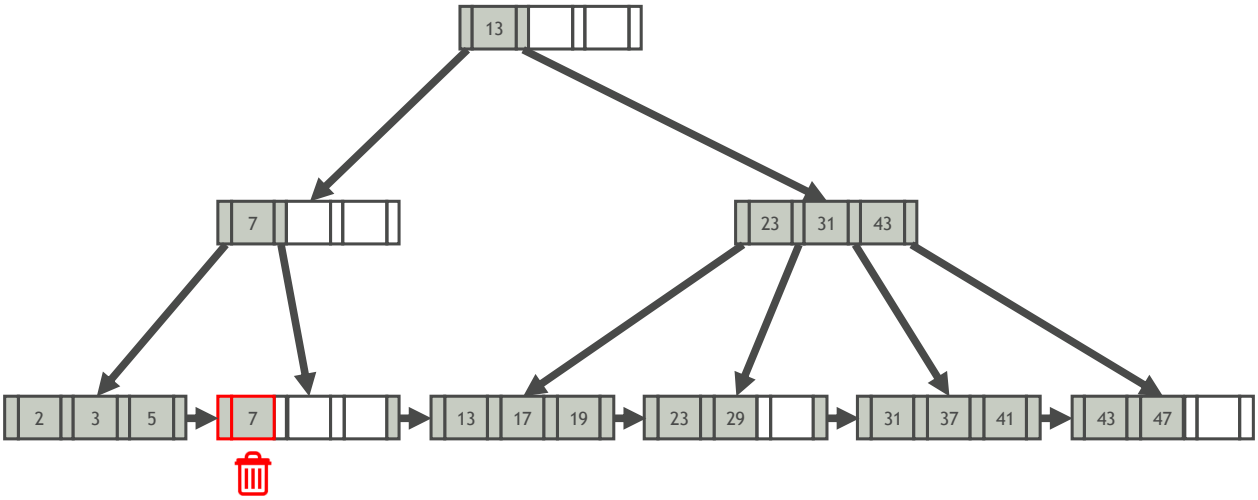
删除索引项(续)

使叶节点L至少半满的处理方法

1. 尝试从L相邻的兄弟节点借一个索引项，使两者均至少半满(如果L向左右兄弟都能借，则优先向左兄弟借)
2. 如果借不到，则将L与其兄弟节点合并(merge)

删除索引项(续)

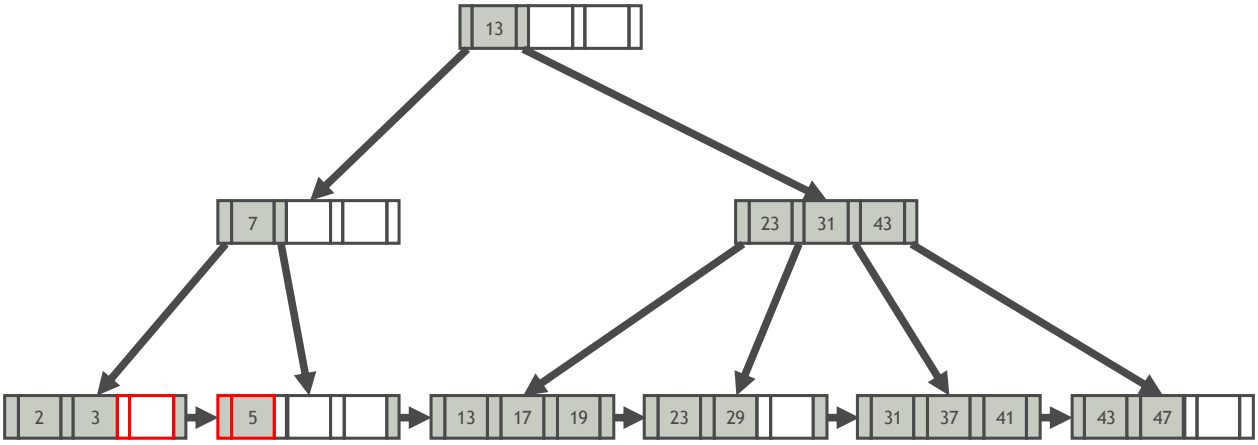
例2: $K = 7$ (需要重分布键)



步骤1/3

删除索引项(续)

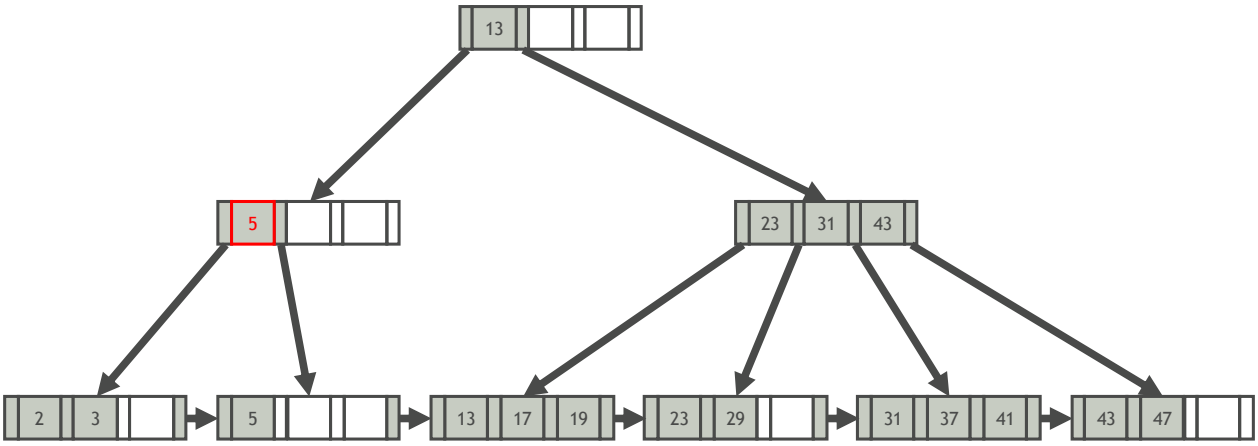
例2: $K = 7$ (需要重分布键)



步骤2/3

删除索引项(续)

例2: $K = 7$ (需要重分布键)



步骤3/3

- 如果一个节点的最小键发生了变化，需要更新其父节点中相应的键
- 如果父节点中最小键也发生了变化，需要递归更新父节点的父节点中相应的键
- ...

删除索引项(续)

如果L借不到，则将L与其兄弟节点合并(merge)

节点合并

1. 如果L与左侧兄弟节点L_1合并，则从L的父节点中删除指向L的指针及相应的键；
   如果L与右侧兄弟节点L_2合并，则从L的父节点中删除指向L_2的指针及相应的键
   如果L与左右兄弟都能合并，则优先与左兄弟合并
2. 如果L的父节点N至少半满，则完成合并；否则，处理N，使N至少半满
   - 如果N是根节点，且N中只有一个指针，则删除N
   - 如果N是内节点，则处理N，使N至少半满

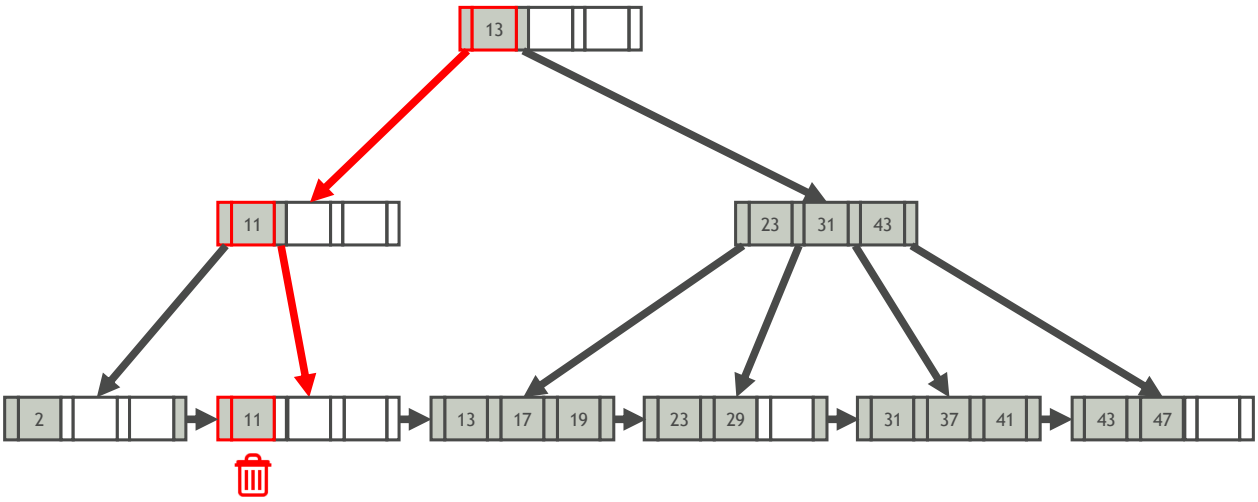
删除索引项(续)

使内节点N至少半满的处理方法

1. 尝试从N相邻的兄弟节点借一个指针及键，使两者均至少半满
2. 如果借不到，则将N与其兄弟节点合并(merge)

删除索引项(续)

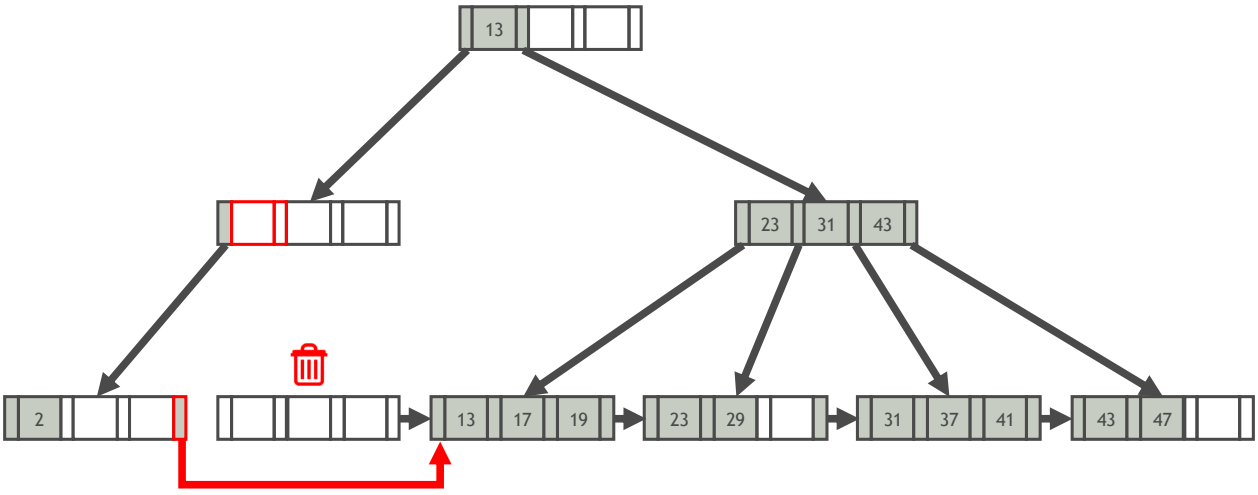
例3: $K = 11$ (需要合并节点)



步骤1/4

删除索引项(续)

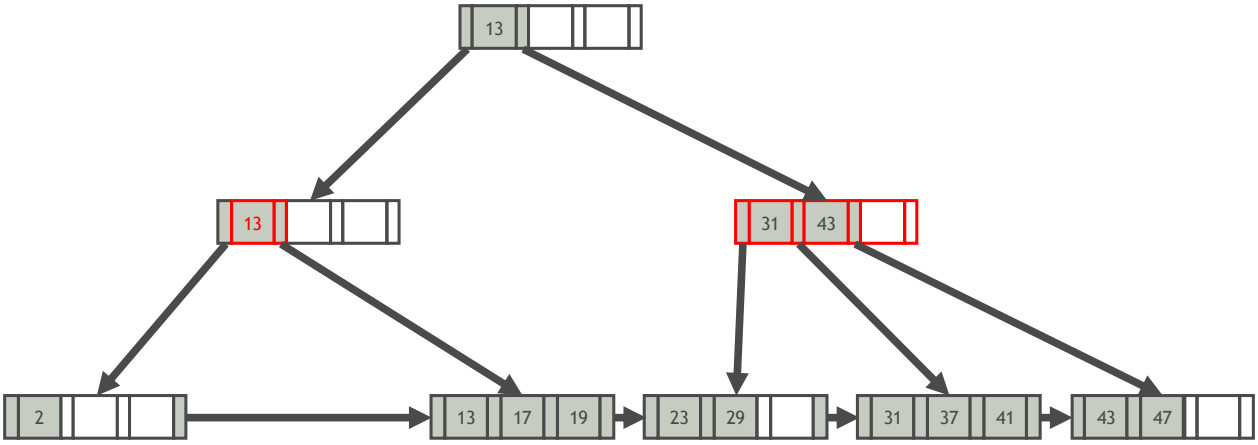
例3: $K = 11$ (需要合并节点)



步骤2/4

删除索引项(续)

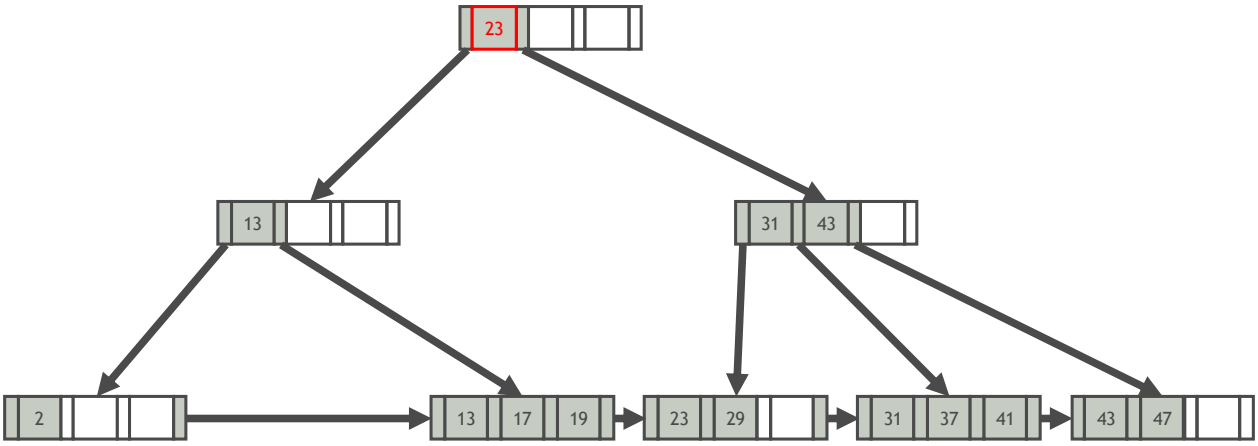
例3: $K = 11$ (需要合并节点)



步骤3/4

删除索引项(续)

例3: $K = 11$ (需要合并节点)



步骤4/4

- 如果一个节点的最小键发生了变化，需要更新其父节点中相应的键
- 如果父节点中最小键也发生了变化，需要递归更新父节点的父节点中相应的键
- ...

Rucbase中B+树插入函数

```
bool IxIndexHandle::delete_entry(
const char *key, Transaction *transaction);

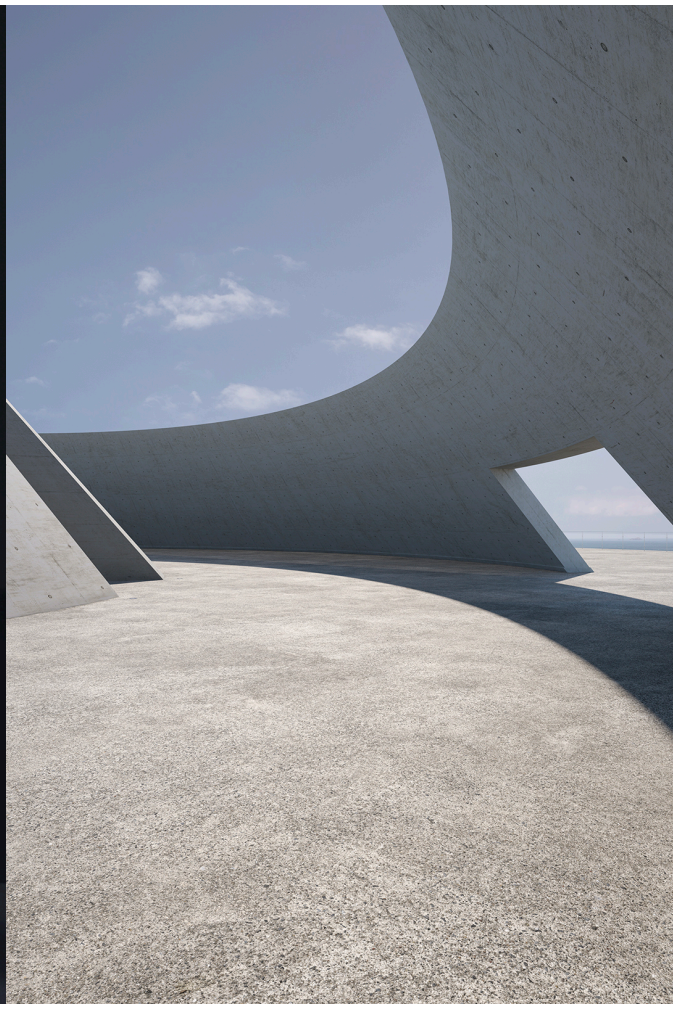
bool IxIndexHandle::CoalesceOrRedistribute(
IxNodeHandle *node, Transaction *transaction);

bool IxIndexHandle::AdjustRoot(IxNodeHandle *old_root_node);

void IxIndexHandle::Redistribute(
IxNodeHandle *neighbor_node, IxNodeHandle *node,
IxNodeHandle *parent, int index);

bool IxIndexHandle::Coalesce(
IxNodeHandle **neighbor_node, IxNodeHandle **node,
IxNodeHandle **parent, int index,
Transaction *transaction);
```


B+树性能优化



键压缩(Key Compression)

对键进行压缩，尽可能减少键的长度

- 从B+树中查找一个索引项所需的磁盘I/O数= B+树的高度≈ $\log_{fan_out}$(# of index entries)
- 索引键越长$\Longrightarrow$ 扇出数越小$\Longrightarrow$ B+树越高$\Longrightarrow$ 查询时间越长

前缀压缩(Prefix Compression)

- 同一叶节点中的键很可能具有相同的前缀(prefix)
- 提取键的公共前缀，只存储每个键的后缀(suffix)

Example (前缀压缩)

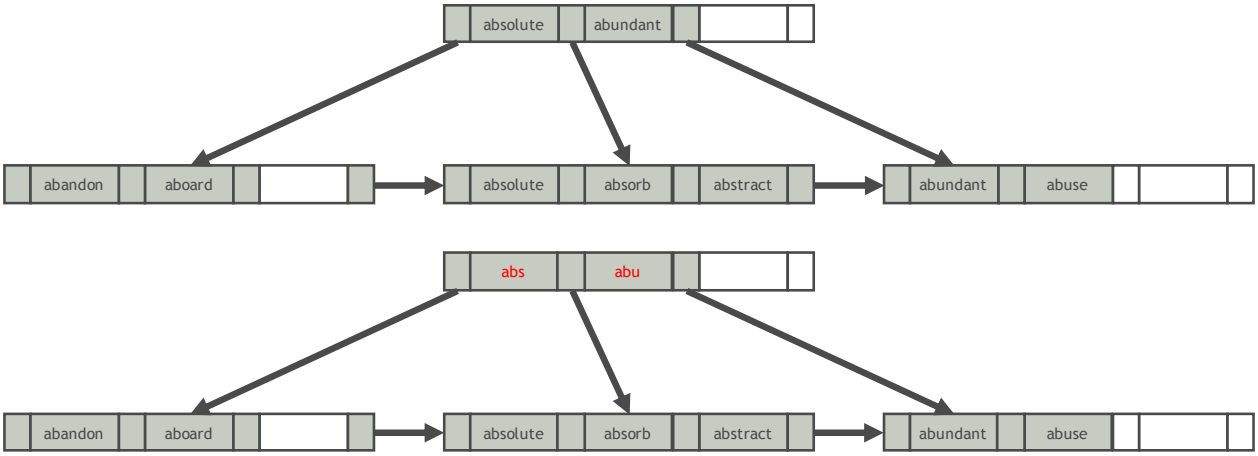
Microphone	Microsoft	Microwave

↓ 前缀压缩

前缀: Micro | phone | soft | wave |

后缀截断(Suffix Truncation)

- 内节点中的键仅用于导航
- 不需要在内节点中存储整个键
- 在保证正确导航的前提下，只需存储每个键的最短前缀即可



批量加载(Bulk Loading)

在一组页中的索引项上建立B+树

自顶向下的方法

- 从一棵空的B+树开始，每次插入一个索引项
- 缺点: 插入每个索引项都需要从根节点向下走到叶节点

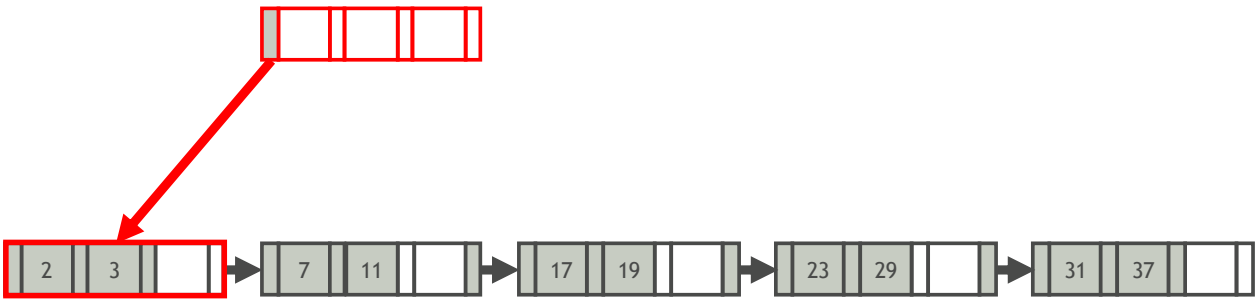
批量加载(续)

自底向上的方法

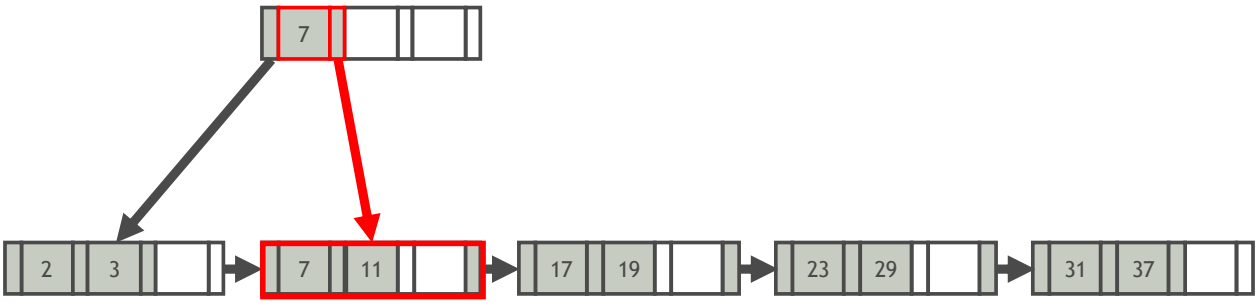
1. 使用外存归并排序算法对所有页中的索引项排序
2. 将每个页作为一个叶节点，建立叶节点链表
3. 创建一个空的内节点作为根，并插入一个指针，指向第一个叶节点
4. 对叶节点链表中的下一个叶节点L，向叶节点上层最右边的内节点插入L中最小的键及指向L的指针；如果内节点溢出，则分裂
5. 重复第4步，直至所有叶节点都插入B+树为止

批量加载(续)

例: 在排好序的键2, 3, 7, 11, 17, 19, 23, 29, 31, 37上建立B+树

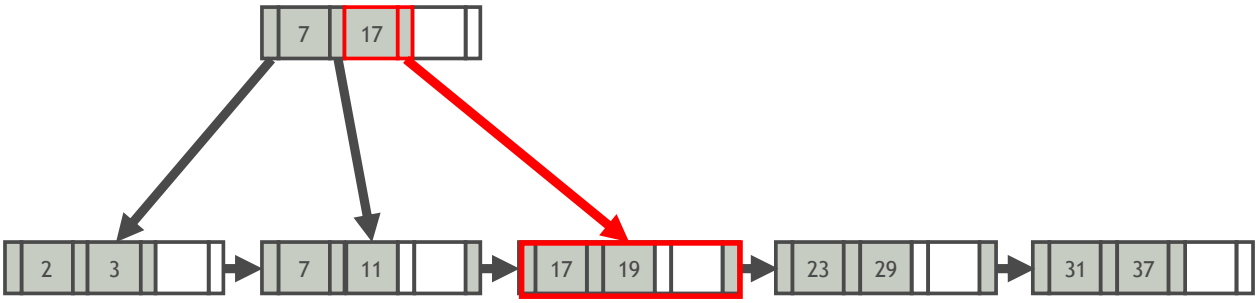


步骤1/5

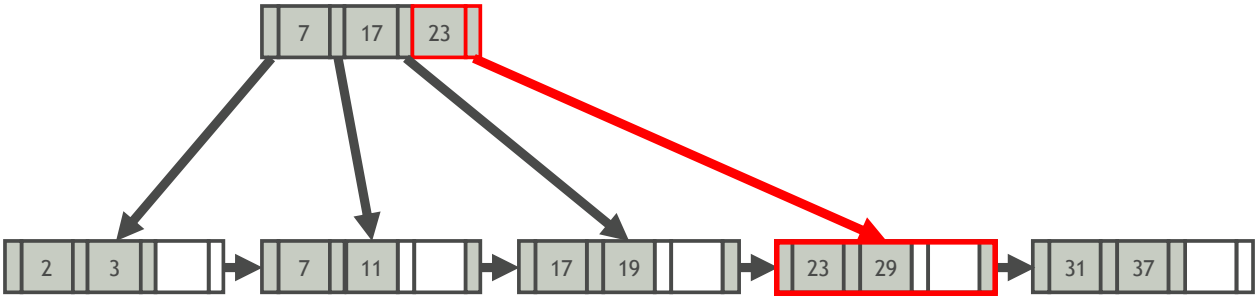


步骤2/5

批量加载(续)

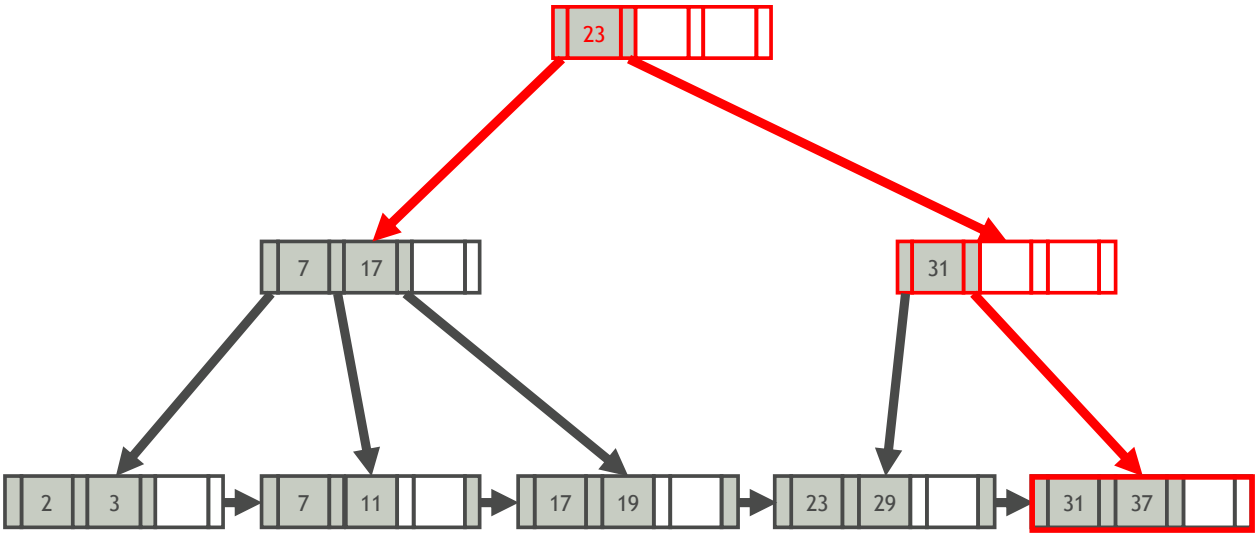


步骤3/5



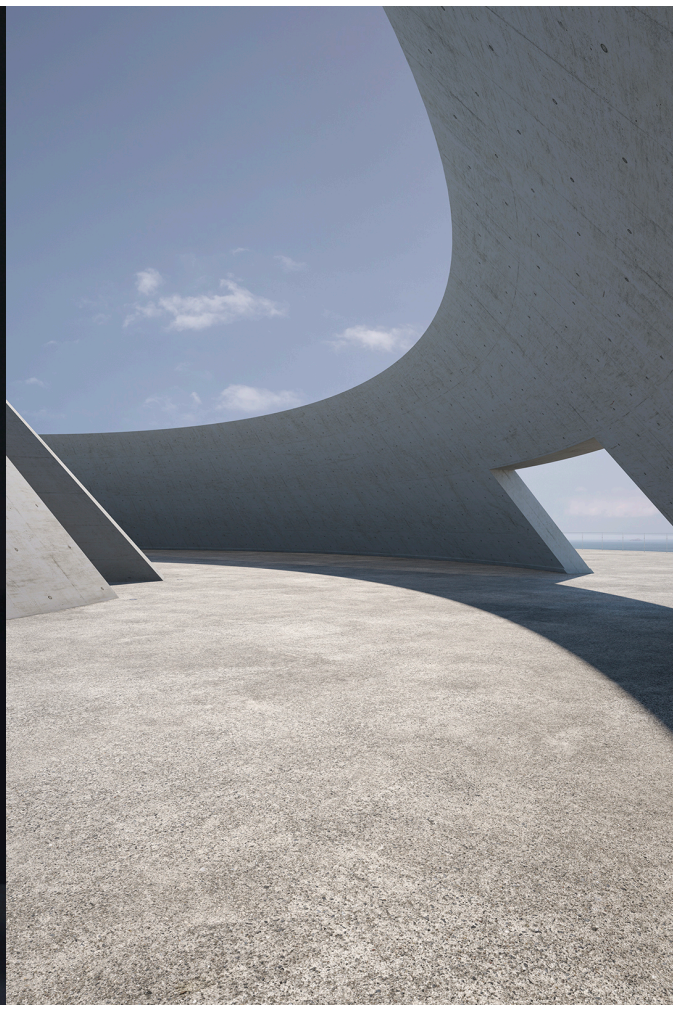
步骤4/5

批量加载(续)



步骤5/5

B+树并发访问

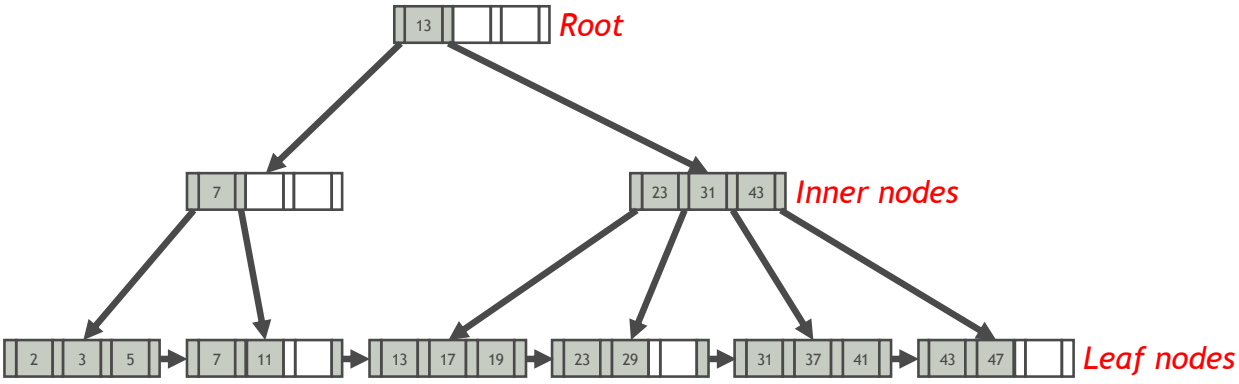


B+树并发访问

多线程并发读写B+树可能产生错误

Example (并发读写相同叶节点可能产生错误)

- 线程1：查找$K = 11$
- 线程2：插入$K = 10$

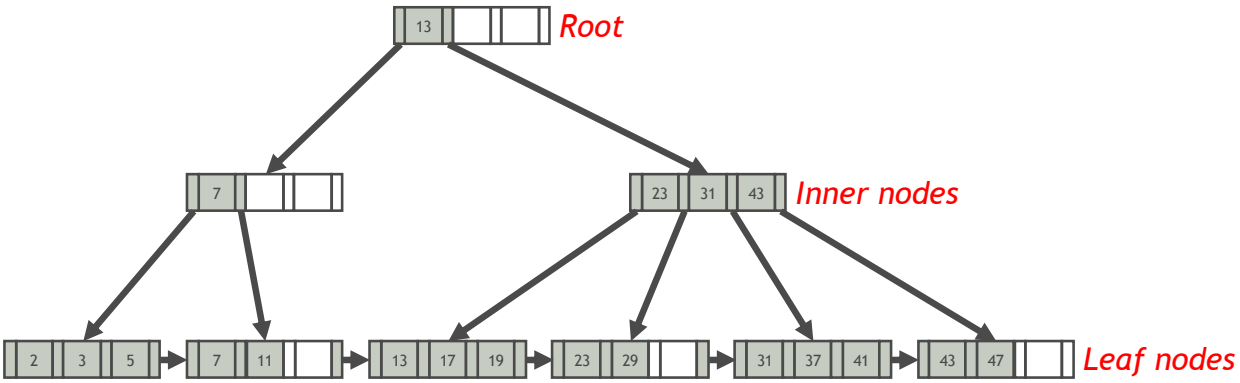


B+树并发访问

多线程并发读写B+树可能产生错误

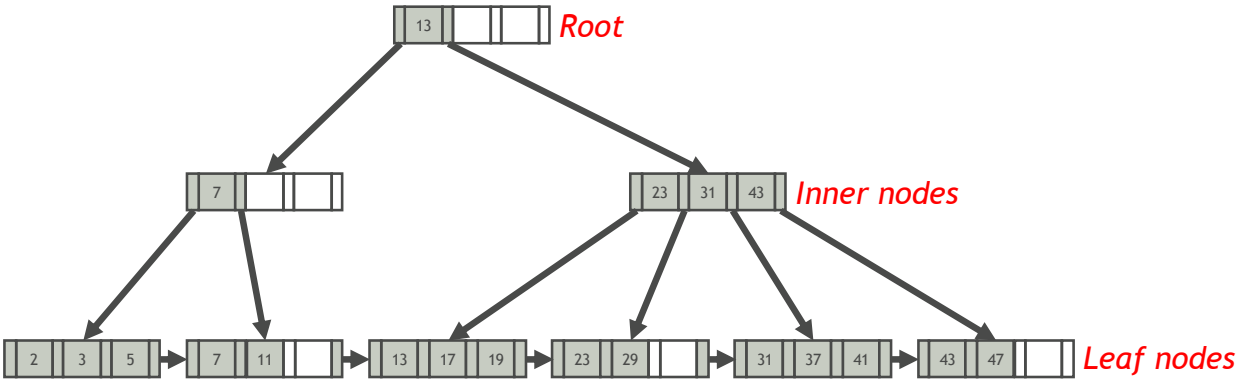
Example (并发读写不同叶节点也可能产生错误)

- 线程1：查找$K = 5$
- 线程2：删除$K = 7$



B+树并发访问

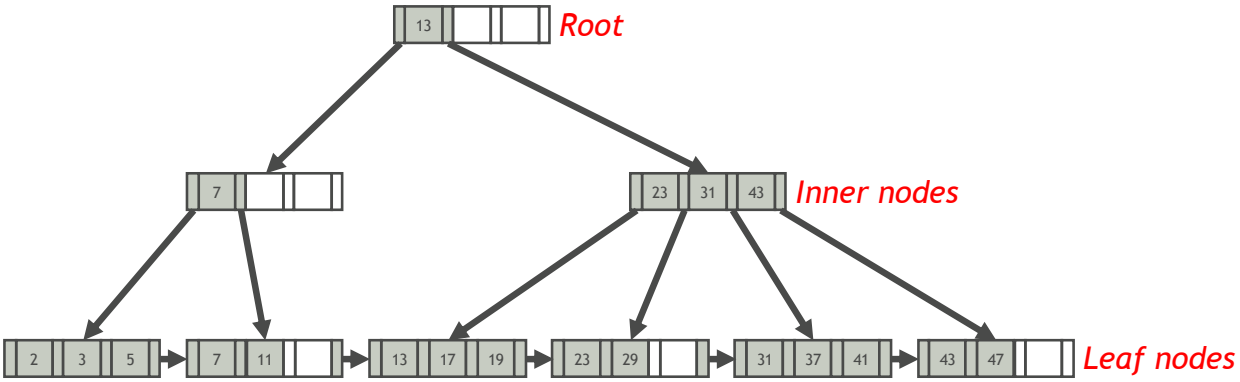
- 粗粒度并发
- 细粒度并发（蟹行协议）



B+树的粗粒度并发访问

每个线程执行下列步骤

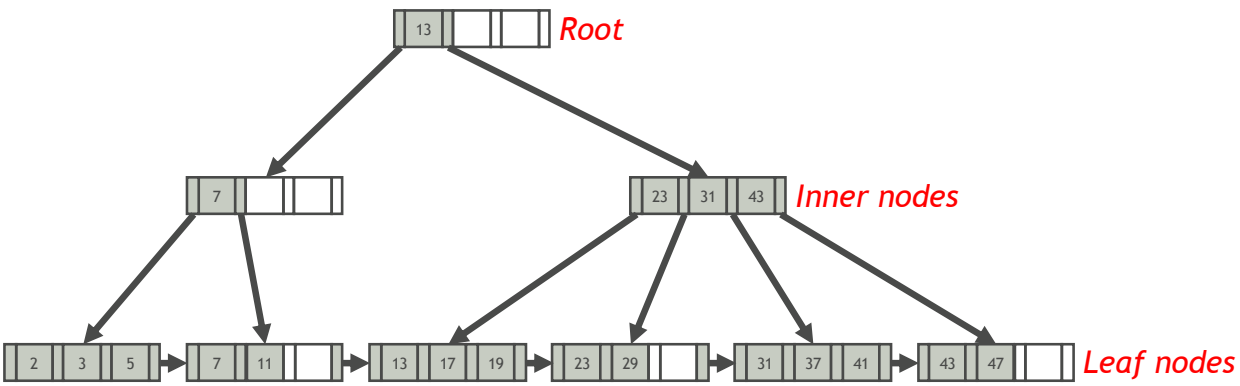
1. 请求对根节点加锁（执行读操作则加读锁，执行写操作则加写锁）
2. 如果加锁成功，则继续访问B+树；否则，等待，直至获得根节点的锁
3. 释放根节点上的锁



B+树的粗粒度并发访问

Example (B+树的粗粒度并发读写)

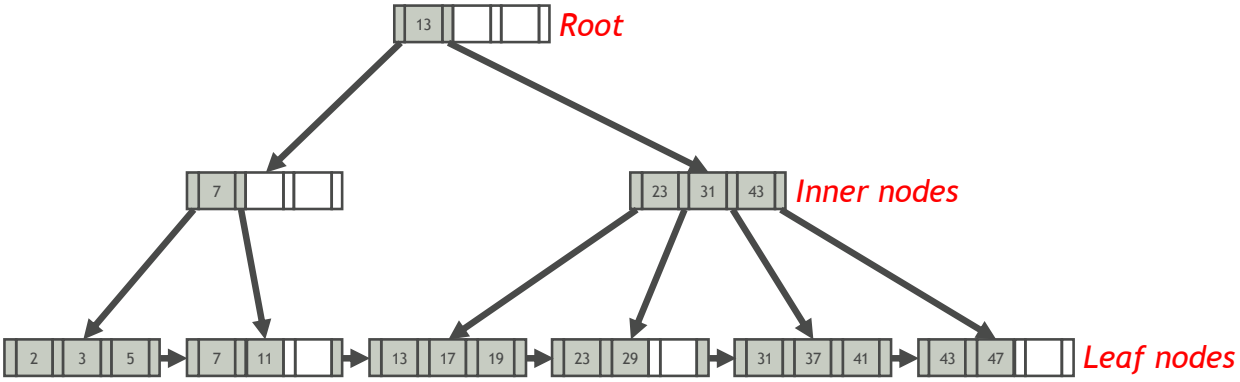
- 线程1：查找$K = 5$
- 线程2：删除$K = 7$



B+树的细粒度并发访问（蟹行协议）

每个线程在自上而下查找叶节点时，只对可能发生冲突的节点加锁

1. 请求对当前节点加锁（执行读操作则加读锁，执行写操作则加写锁）
2. 如果加锁成功，则继续；否则，等待，直至获得当前节点的锁
3. 如果当前节点安全（不会发生分裂或合并），则释放其祖先节点上的锁（冲突不会发生在这些祖先节点上）
4. 继续访问相应的儿子节点，执行步骤1–3

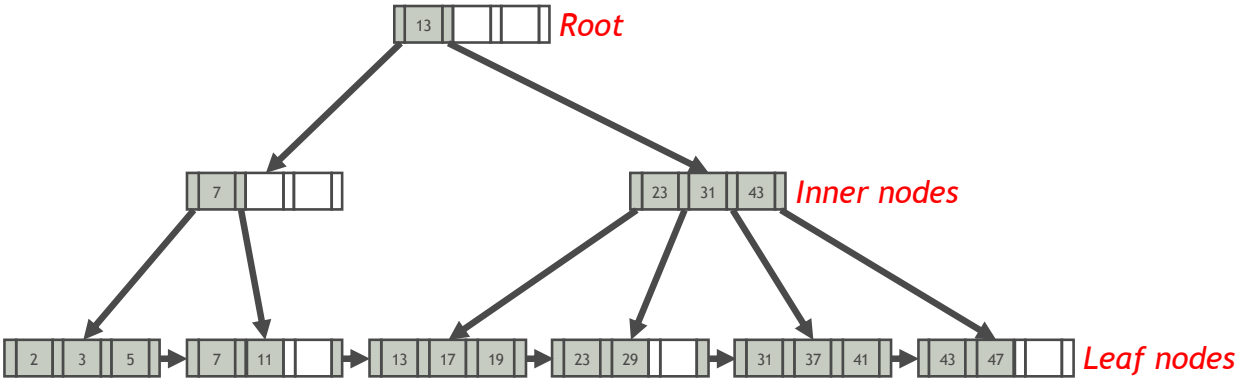


B+树的细粒度并发访问（蟹行协议）

每个线程在自上而下查找叶节点时，只对可能发生冲突的节点加锁

Example (B+树细粒度并发读写的加锁过程)

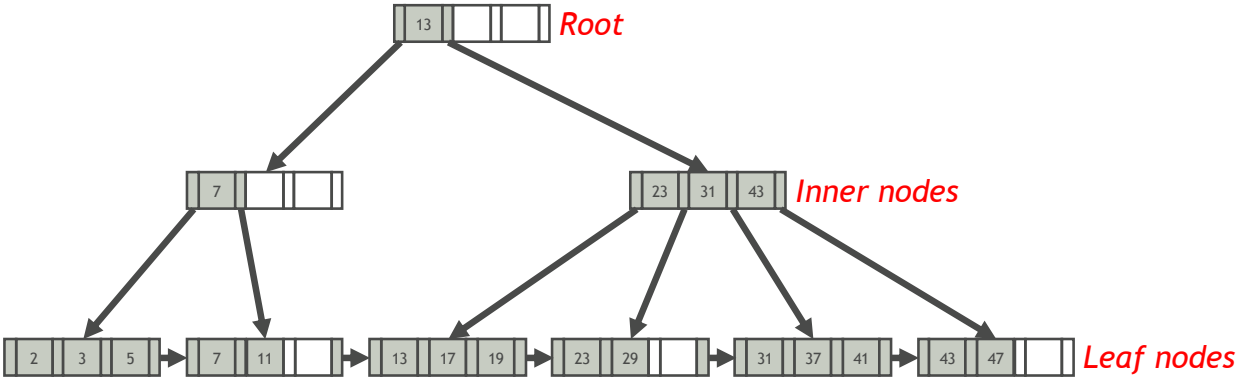
- 线程1：查找$K = 43$
- 线程2：插入$K = 32$



B+树的细粒度并发访问（蟹行协议）

在完成对叶节点的读写后，执行下列解锁步骤

1. 释放当前节点上的锁
2. 如果需要对当前节点的父节点进行分裂或合并，则在分裂或合并后，释放父节点上的锁；否则，释放当前节点祖先节点上的锁
3. ……

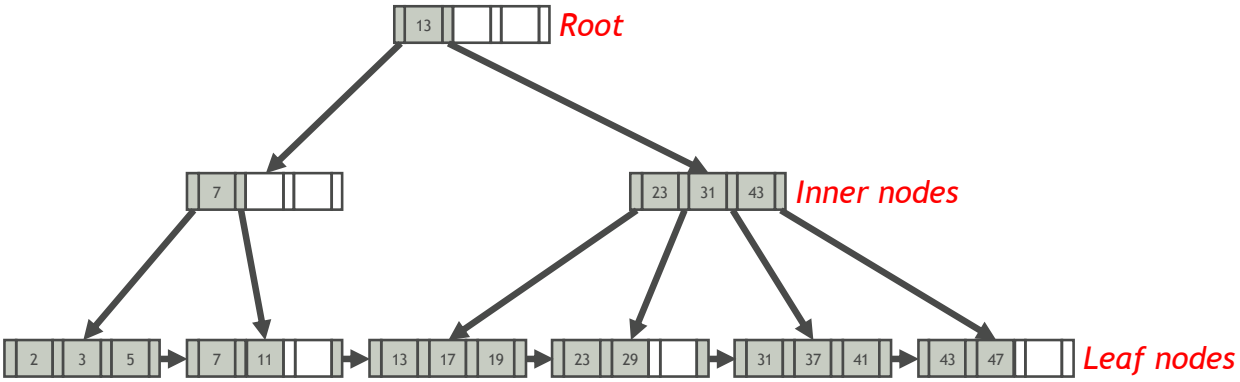


B+树的细粒度并发访问（蟹行协议）

在完成对叶节点的读写后，执行下列解锁步骤

Example (B+树细粒度并发读写的解锁过程)

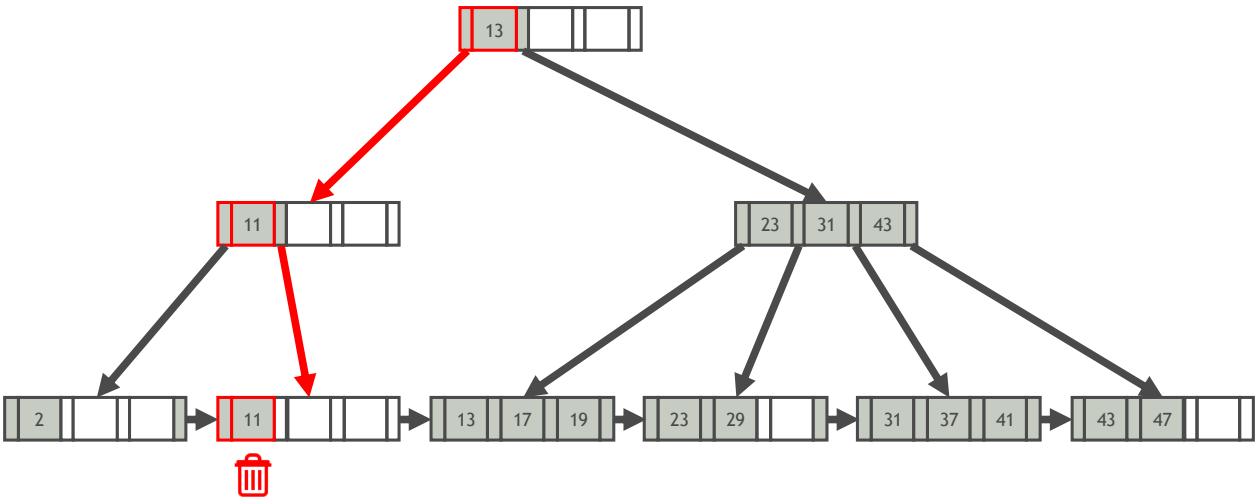
- 线程1：查找$K = 43$
- 线程2：插入$K = 32$



B+树的细粒度并发访问（蟹行协议）

Example (B+树细粒度并发读写过程)

- 线程1：删除$K = 11$
- 线程2：插入$K = 32$



讨论

